Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

SISTEMA DE AUTORIZACIÓN DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS MEDIANTE UNA API WEB EN C# CON ASP.NET CORE, PERSISTENCIA EN JSON Y FRONTEND EN RAZOR PAGES

202307900 – Jonathan Jeanfranco Chirix Santizo

Resumen

La Superintendencia de Administración Tributaria (SAT) de Guatemala impulsa la factura electrónica como mecanismo de control fiscal. Este ensayo expone el diseño y la construcción de un servicio web que autoriza Documentos Tributarios Electrónicos (DTE). La solución se compone de una API desarrollada en C# con ASP.NET Core, que recibe mensajes JSON por medio del protocolo HTTP, y de un frontend en Razor Pages que funciona como simulador para enviar archivos, consultar estadísticas, graficar resultados y generar reportes en PDF. Se describen la arquitectura por capas adoptada, el modelado orientado a objetos con tipos de dato abstracto propios, la validación del NIT mediante el algoritmo de módulo 11, la extracción de fechas con expresiones regulares, la generación del número de autorización con un correlativo diario y la persistencia en archivos JSON. Se concluye que separar responsabilidades en capas y encapsular las reglas fiscales dentro del dominio produce un sistema verificable, fácil de probar desde clientes como Postman y preparado para crecer sin modificar los contratos de comunicación.

Palabras clave

API web, ASP.NET Core, JSON, programación orientada a objetos, expresiones regulares.

Abstract

The Superintendence of Tax Administration (SAT) of Guatemala promotes electronic invoicing as a tax control mechanism. This essay presents the design and construction of a web service that authorizes Electronic Tax Documents (DTE). The solution consists of an API developed in C# with ASP.NET Core, which receives JSON messages through the HTTP protocol, and a Razor Pages frontend that works as a simulator to send files, query statistics, chart results and generate PDF reports. The essay describes the layered architecture, the object-oriented model built with custom abstract data types, the validation of the tax identification number (NIT) with the modulo 11 algorithm, the extraction of dates with regular expressions, the generation of the authorization number with a daily sequence and the persistence in JSON files. It concludes that separating responsibilities into layers and encapsulating the tax rules inside the domain produces a verifiable system that is easy to test from clients such as Postman and ready to grow without changing its communication contracts.

Keywords

Web API, ASP.NET Core, JSON, object-oriented programming, regular expressions.

Introducción

La facturación electrónica obliga a que cada documento tributario sea validado y autorizado por la administración tributaria antes de tener validez. Esto convierte a la autorización en un servicio que debe estar disponible por Internet, responder de forma inmediata y conservar la información para auditorías.

El presente ensayo describe la solución desarrollada para el Proyecto 3 de Introducción a la Programación y Computación 2: una API en C# que actualiza el Registro Tributario Unificado (RTU), autoriza facturas y responde consultas, junto con una aplicación web que la consume. Se busca responder dos preguntas: ¿cómo organizar el software para que las reglas fiscales sean correctas y fáciles de verificar? y ¿qué estructuras y algoritmos permiten procesar y consultar la información de forma eficiente sin depender de una base de datos?

Desarrollo del tema

El desarrollo se organiza en siete subtemas que siguen el camino de un mensaje: desde que llega a la API hasta que se convierte en un reporte para el usuario. Los diagramas completos de clases, actividades y despliegue se presentan en los apéndices A, B y C.

a. El problema: autorizar un DTE

El emisor de una factura envía un mensaje con la fecha, una referencia interna, el NIT del emisor, el NIT del receptor, el valor, el IVA y el total. La SAT debe comprobar que ambos NIT sean válidos y estén inscritos en el RTU, que el IVA y el total estén bien calculados y que la referencia no se repita. Si todo es correcto, devuelve un número único de autorización con la forma *yyyymmdd########*, cuyo correlativo reinicia en 1 cada día.

Además del servicio de autorización, el sistema debe actualizar el RTU, generar reportes por rango de fechas con el detalle de errores y listar las facturas emitidas o recibidas por un contribuyente. Toda la comunicación usa mensajes JSON con una estructura fija que no puede modificarse (Bray, 2017).

b. Arquitectura de la solución

La solución está formada por tres proyectos de .NET 10. **Sat.Api** (Servicio 2) es la API web que contiene toda la lógica. **Sat.Web** (Programa 1) es el frontend en Razor Pages. **Sat.Compartido** es una biblioteca con los contratos JSON que usan ambos, lo que garantiza que el cliente y el servidor hablen exactamente el mismo idioma.

Dentro de la API se aplicó una arquitectura por capas (Fowler, 2002). Los *controllers* reciben la petición HTTP y la traducen; los *servicios* coordinan cada caso de uso; el *dominio* contiene las entidades, los TDA y las reglas fiscales; y la capa de *datos* guarda y recupera la información. Cada capa solo conoce a la de abajo, como muestra la figura 1.

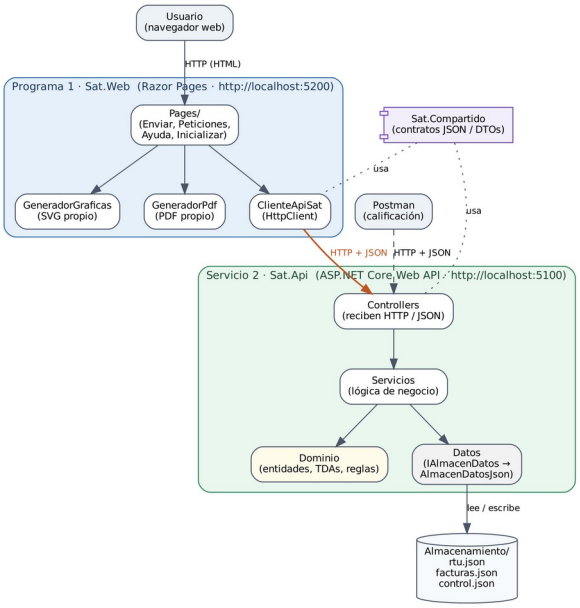


Figura 1. Arquitectura general de la solución por capas.

Fuente: elaboración propia, 2026.

La API sigue el estilo de recursos sobre HTTP propuesto por Fielding (2000): cada operación tiene una ruta y un verbo. La tabla I resume los métodos implementados.

Tabla I.

Métodos expuestos por la API (Servicio 2).

MÉTODO Y RUTA	FUNCIÓN
POST /api/rtu	Crea o actualiza contribuyentes del RTU
GET /api/rtu	Lista los contribuyentes registrados
POST /api/procesar	Procesa solicitudes de autorización
GET /api/consultadatos	Facturas emitidas o recibidas por NIT
GET /api/resumeniva	IVA emitido y recibido por NIT en una fecha
GET /api/resumenrango	Valor autorizado por fecha (total o sin IVA)
GET /api/reporte	Reporte de errores y estadísticas por fecha
POST /api/inicializar	Devuelve el sistema al estado inicial

Fuente: elaboración propia, 2026.

Las respuestas de error también se controlaron: si el JSON viene mal formado o una fecha no tiene el formato *yyyy-MM-dd*, la API responde con el código 400 y un mensaje en español, en lugar del mensaje técnico en inglés que genera el *framework* por defecto.

c. Modelo orientado a objetos y TDA propios

El dominio se modeló con clases que protegen sus propias reglas. **Contribuyente** guarda un NIT ya normalizado y su nombre. **Factura** conoce su fecha de emisión, sus montos y una lista de errores del tipo enumerado **TipoError**; solo puede pasar a aprobada si no tiene errores, y en ese momento recibe su código de autorización. Este encapsulamiento evita estados inválidos, como una factura con errores y con código a la vez.

Sobre las colecciones de C# se construyeron tres TDA propios que ocultan cómo se guardan los datos:

- **RegistroTributario:** envuelve un diccionario cuya llave es el NIT, por lo que verificar si un contribuyente existe toma tiempo constante.
- **LibroFacturas:** mantiene la lista de facturas y un conjunto (*HashSet*) con las referencias aprobadas para detectar duplicados de inmediato.
- **ControlCorrelativo:** recuerda la fecha actual y el último número emitido para generar los códigos de autorización.

Las clases de dominio no se serializan directamente. Para los archivos existen clases de *registro* y para la comunicación existen los contratos de Sat.Compartido; la conversión entre ellas ocurre en la capa de datos y en los servicios. El diagrama de clases completo se encuentra en el apéndice A.

d. Algoritmos principales

Validación del NIT. Se separa el último carácter como verificador. Cada dígito restante se multiplica por su posición, contando desde la derecha y comenzando en 2, y los productos se suman:

$$S = \Sigma_{i=2}^{n}\ d_i \cdot i \qquad (1)$$

Con esa suma se obtiene el dígito verificador esperado:

$$V = (11 - S \bmod 11) \bmod 11 \qquad (2)$$

donde:

S = suma de los productos

d_i = dígito que ocupa la posición i desde la derecha

n = cantidad de caracteres del NIT

V = dígito verificador; si vale 10 se representa con K

El NIT es válido cuando *V* coincide con el último carácter. Antes de validar se quitan espacios y guiones, y se rechazan bases con caracteres que no sean dígitos.

Reglas fiscales. Los montos se manejan con el tipo *decimal* para evitar errores de redondeo binario, y el redondeo se hace hacia arriba en el punto medio (*MidpointRounding.AwayFromZero*):

$$IVA = redondear(VALOR \cdot 0.12,\ 2) \qquad (3)$$

El total esperado es *VALOR* + *IVA*, y cualquier diferencia se registra como error.

Extracción de la fecha. El campo TIEMPO puede traer texto adicional, como «Guatemala, 15/01/2025 15:25 hrs.». En lugar de recortar posiciones fijas se usa la expresión regular *(\d{1,2})/(\d{1,2})/(\d{4})(?:\s+(\d{1,2}):(\d{2}))?*, que localiza la fecha y la hora en cualquier parte del texto. Los grupos capturados se convierten a números y se valida que formen una fecha real.

Número de autorización. Si la fecha cambió desde la última autorización, el correlativo vuelve a 0; luego se incrementa y se calcula:

$$código = F \cdot 10^{8} + c \qquad (4)$$

donde:

F = fecha numérica yyyymmdd (por ejemplo, 20261022)

c = correlativo del día, de 1 a 99 999 999

Al usar aritmética entera en un *long*, los ceros de relleno aparecen solos: el primer código del día es 2026102200000001.

Proceso de autorización. Cada factura del lote pasa por todas las validaciones y acumula sus errores en lugar de detenerse en el primero; así el reporte puede contar cada tipo de error. Si no hay errores, recibe su código y su referencia se marca como aprobada. Al terminar el lote se guardan los archivos y se arma la respuesta con la estructura exacta del enunciado. Los diagramas de actividades de estos algoritmos están en el apéndice B.

e. Persistencia y consistencia de datos

La información se guarda en tres archivos JSON: *rtu.json*, *facturas.json* y *control.json*. Al iniciar, la API los carga en los TDA; después de cada operación que modifica datos, los vuelve a escribir. La interfaz *IAlmacenDatos* separa el dominio del formato de los archivos, de modo que cambiar a una base de datos solo exigiría una nueva implementación.

Como ASP.NET Core atiende peticiones en paralelo, las operaciones de escritura se protegen con un bloqueo (*lock*). Esto impide que dos lotes simultáneos obtengan el mismo correlativo o aprueben dos veces la misma referencia.

f. Consultas, gráficas y reportes en el frontend

Las consultas se resolvieron con algoritmos propios. Para el reporte por rango se crea un acumulador por cada día; cada factura se ubica en su día, suma a sus contadores de errores y, si fue aprobada, agrega los NIT a dos conjuntos cuyo tamaño final es la cantidad de emisores y receptores distintos. El mismo patrón se usa para los resúmenes de IVA por NIT y de valores por fecha.

El frontend consume la API por medio de un *HttpClient* tipado (Microsoft, 2025). Cada petición es un formulario GET que muestra los resultados en tablas. Las gráficas se generan como SVG con una clase propia que calcula una escala legible para el eje vertical (figura 2).

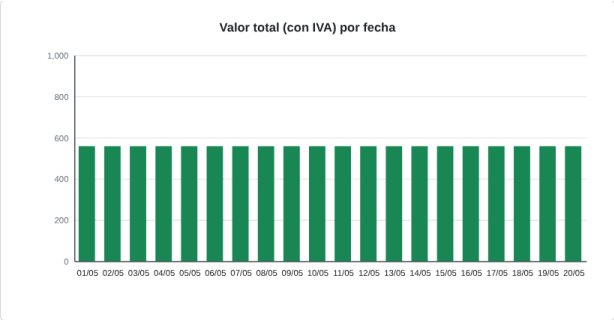


Figura 2. Gráfica del resumen por rango de fechas generada por el frontend.

Fuente: elaboración propia, 2026.

Los reportes en PDF se generan con un escritor propio que produce el archivo directamente con fuentes estándar, saltos de página y encabezados de tabla repetidos. De esta forma no se depende de bibliotecas externas.

g. Despliegue y pruebas

Ambas aplicaciones se ejecutan sobre el servidor web Kestrel: la API en *http://localhost:5100* y el frontend en *http://localhost:5200*. Los archivos de datos residen en la carpeta *Almacenamiento* del servidor. El diagrama de despliegue se presenta en el apéndice C.

Para verificar la solución se construyó un proyecto de consola con 108 pruebas de las reglas y los TDA, y se ejecutaron los archivos de prueba oficiales desde Postman y desde el frontend. Los resultados coincidieron con los esperados en todos los casos, incluidos los mensajes mal formados y los NIT inválidos.

Conclusiones

Separar la solución en capas y en un proyecto de contratos compartidos permitió que cada parte tenga una sola responsabilidad. Las reglas fiscales viven en el dominio y pueden probarse sin levantar el servidor ni usar HTTP.

Los TDA construidos sobre diccionarios y conjuntos resolvieron los puntos críticos del problema, que son la existencia de un NIT y la unicidad de una referencia, en tiempo constante. Los acumuladores por fecha producen los reportes en una sola pasada sobre las facturas.

Las expresiones regulares y el tipo *decimal* demostraron ser herramientas adecuadas para procesar datos reales: toleran texto adicional en las fechas y evitan diferencias de centavos en el IVA.

Como trabajo futuro, la interfaz de almacenamiento permite migrar a una base de datos y agregar autenticación sin cambiar los controladores. Queda abierta la pregunta de cómo escalar el correlativo diario si varias instancias del servicio atendieran solicitudes al mismo tiempo.

Referencias bibliográficas

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC

8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Microsoft. (2025). *ASP.NET Core documentation*. Microsoft Learn. https://learn.microsoft.com/aspnet/core

Superintendencia de Administración Tributaria. (2026). *Factura electrónica en línea (FEL)*. https://portal.sat.gob.gt

Apéndice A. Diagramas de clases

El modelo se divide en cuatro diagramas por proyecto y capa para que sea legible. Las relaciones entre ellos ocurren por medio de los servicios (API) y del cliente HTTP (frontend).

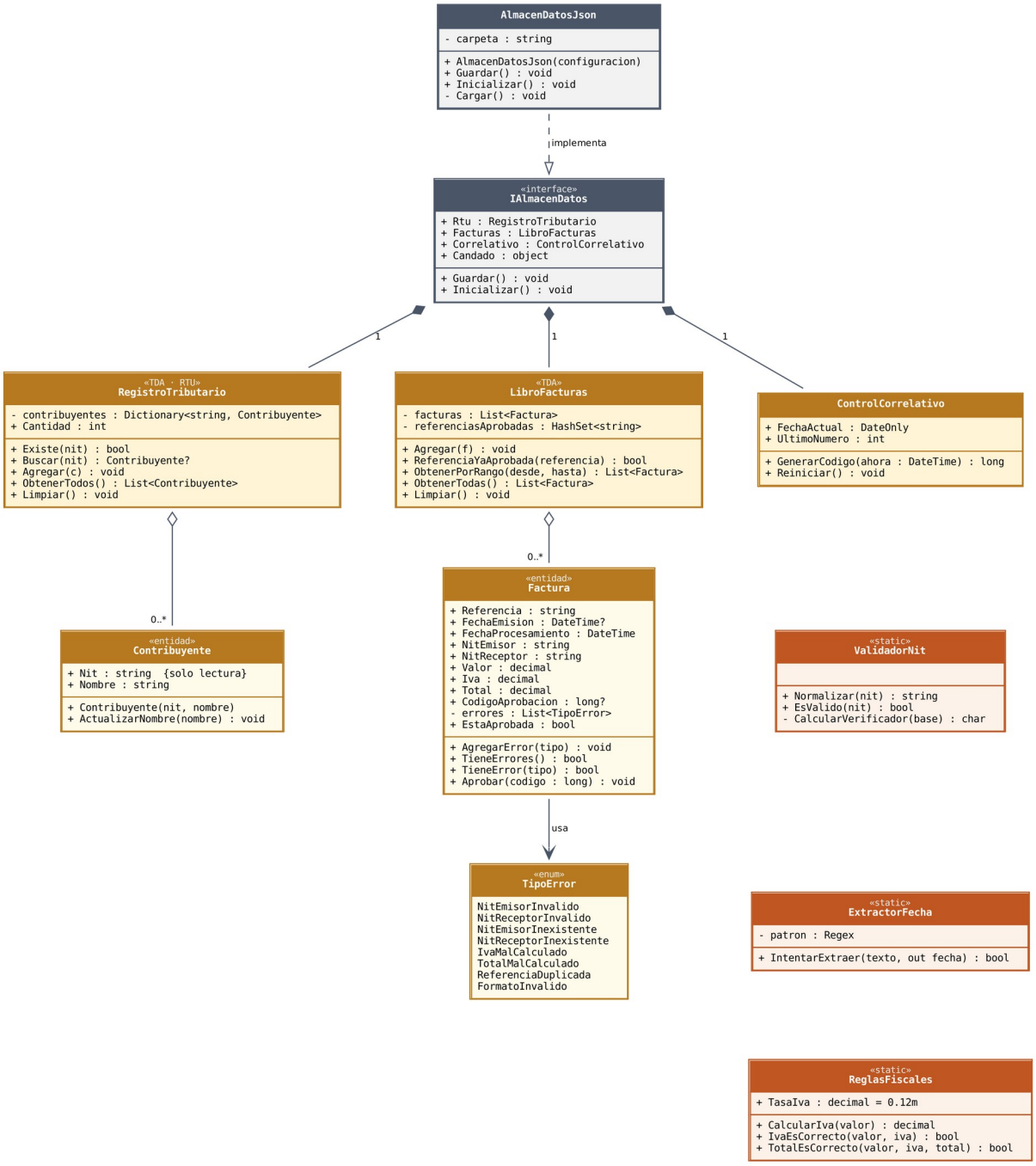


Figura 3. Diagrama de clases del dominio: entidades, TDA y reglas fiscales.

Fuente: elaboración propia, 2026.

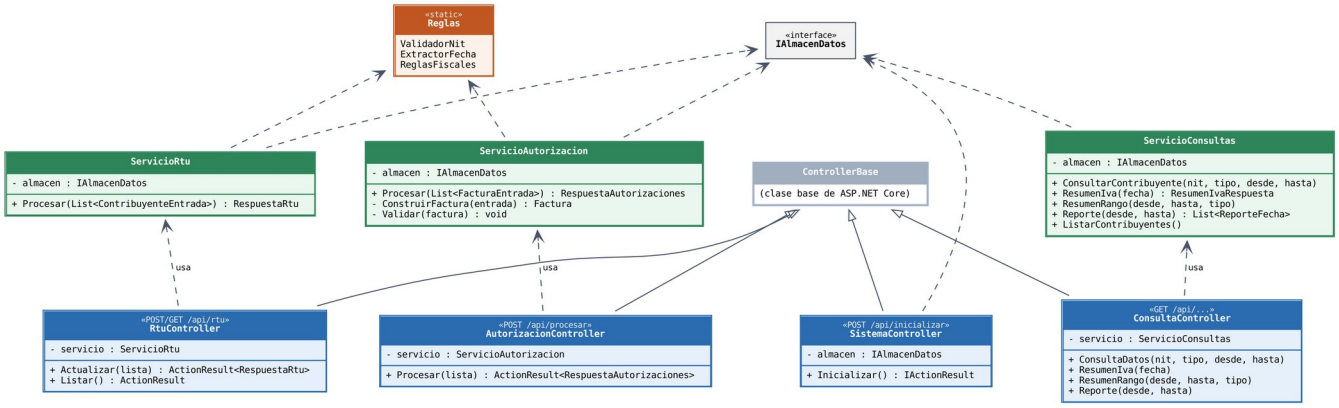


Figura 4. Diagrama de clases de la API: controllers, servicios y capa de datos.

Fuente: elaboración propia, 2026.

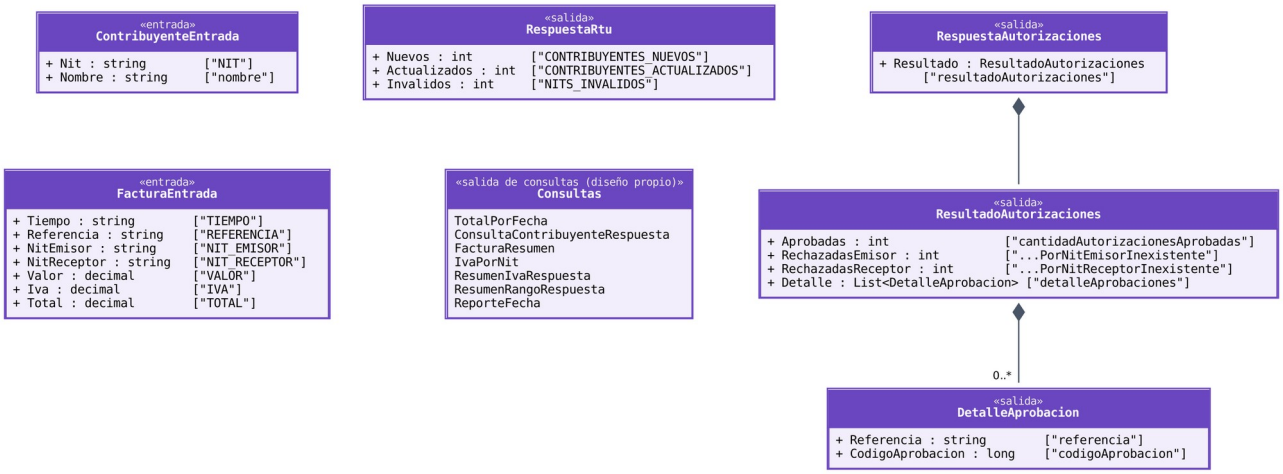


Figura 5. Diagrama de clases de los contratos JSON (Sat.Compartido).

Fuente: elaboración propia, 2026.

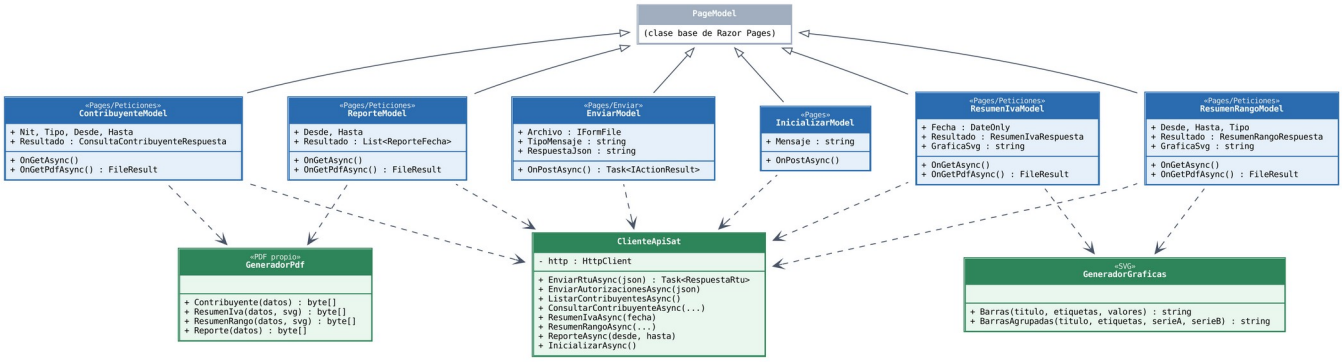


Figura 6. Diagrama de clases del frontend (Sat.Web).

Fuente: elaboración propia, 2026.

Apéndice B. Diagramas de actividades

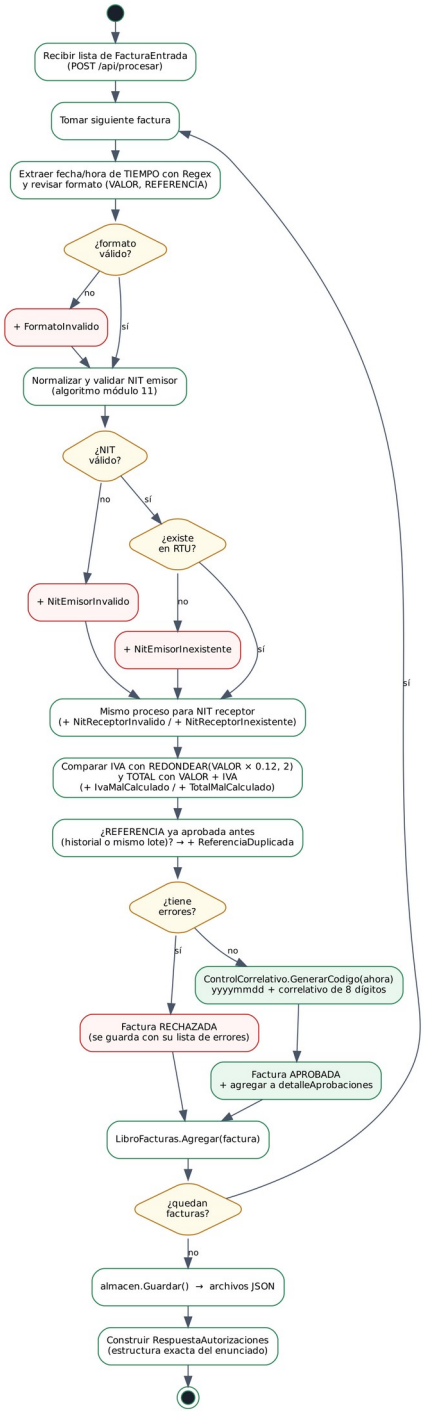


Figura 7. Diagrama de actividades del proceso de autorización de facturas.

Fuente: elaboración propia, 2026.

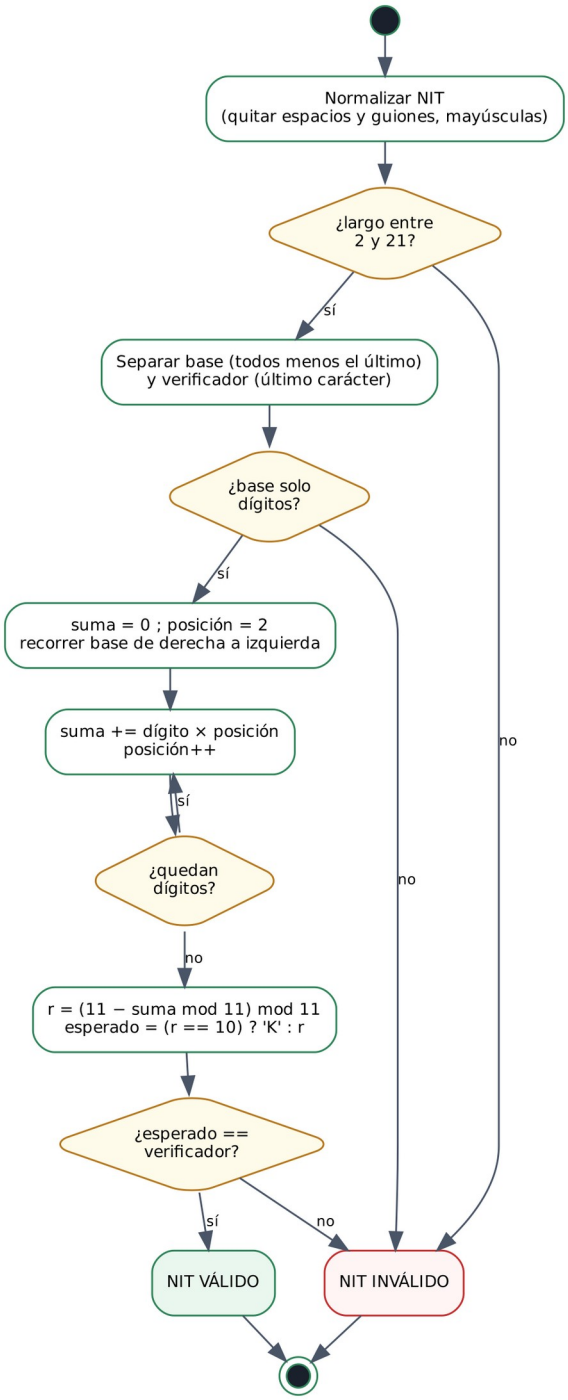


Figura 8. Diagrama de actividades de la validación del NIT (módulo 11).

Fuente: elaboración propia, 2026.

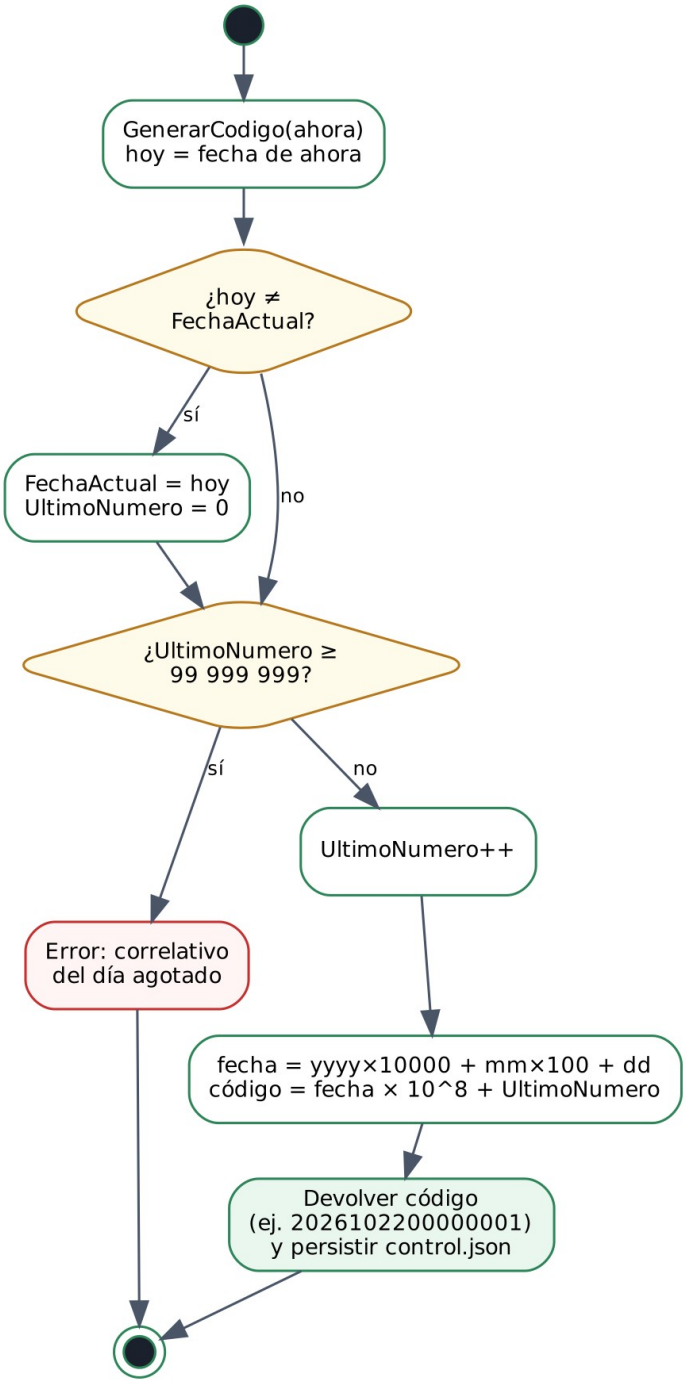


Figura 9. Diagrama de actividades de la generación del número de autorización.

Fuente: elaboración propia, 2026.

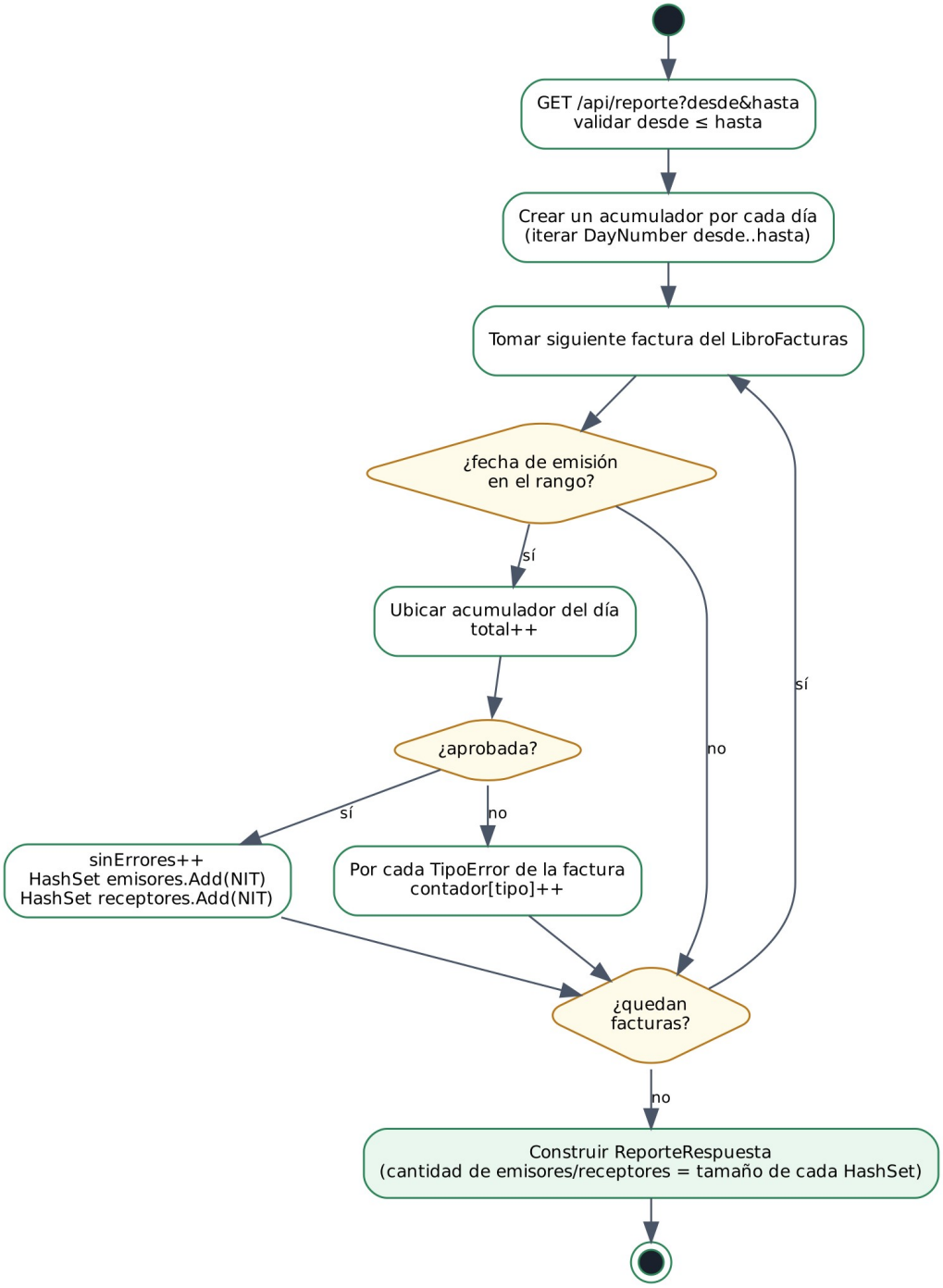


Figura 10. Diagrama de actividades del reporte por rango de fechas.

Fuente: elaboración propia, 2026.

Apéndice C. Diagrama de despliegue

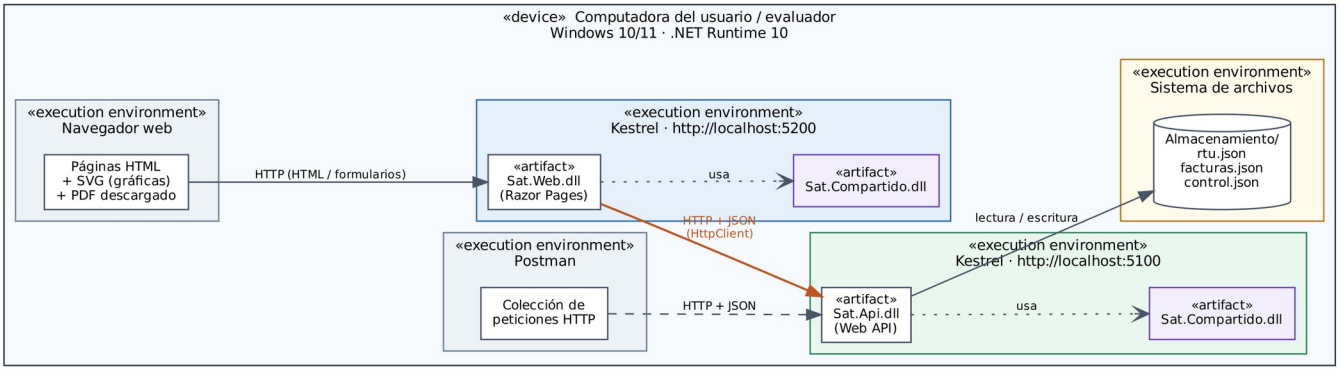


Figura 11. Diagrama de despliegue del software desarrollado.

Fuente: elaboración propia, 2026.